

Secure Digital Document Management System (DMS) for Legal & Investigative Agencies

Architecture, Flow, Tech Stack & Requirement Coverage

1. Project Goal (Restated)

Build a **secure, centralized, intelligent DMS** — backend in **Spring Boot** — for law enforcement, courts, legal departments, and investigative bodies to store, organize, retrieve, and share FIRs, investigation records, witness statements, charge sheets, court filings, evidence, forensic reports, and judgments — while guaranteeing confidentiality, integrity, auditability, and legal validity.

2. High-Level Architecture



CLIENT LAYER

Web Portal (React/Angular) | Mobile App | Admin Console |

HTTPS / TLS 1.3

API GATEWAY LAYER

Spring Cloud Gateway — Auth check, rate limiting, request routing |

APPLICATION SERVICE LAYER (Spring Boot)

Identity & Access Service (Auth/RBAC/ABAC, MFA)	Document Service (Upload, versioning)	Case Management Service (Case linking, workflow, status tracking)
--	--	--

Digital Signature & Blockchain Integrity Svc	Audit & Compliance Service (immutable log)	Search & AI Service (OCR, NLP tagging, full-text/semantic search)
--	---	--

Notification & Workflow Svc (approvals, escalations)	Collaboration Service (comments, shared review)
---	--

DATA LAYER

PostgreSQL (metadata, case data)	Encrypted Object Storage (documents, S3/MinIO)
Redis (cache/session)	Elasticsearch (search index)
	Blockchain ledger (Hyperledger Fabric — hash anchoring)

Design pattern: **microservices** (or a well-modularized monolith to start, split later), each owning its own data where feasible, communicating via REST/gRPC internally and events (Kafka) for async workflows (audit

logging, notifications, search indexing).

3. Core Modules & What They Do

Identity & Access Management (IAM)

Role-Based + Attribute-Based Access Control (RBAC/ABAC): roles like Investigating Officer, Prosecutor, Judge, Clerk, Admin.

Multi-Factor Authentication (MFA), SSO via OAuth2/OIDC.

Fine-grained document/case-level permissions (who can view/edit/share a specific FIR).

Document Service

Upload, download, versioning (every edit creates a new immutable version).

Metadata tagging (case number, document type, date, officer ID, jurisdiction).

File encryption at rest (AES-256) and in transit (TLS).

Digital Signature & Integrity Service

PKI-based digital signatures (e.g., using eIDAS/IT Act-compliant certs) so filings/reports are legally attributable.

Document hash (SHA-256) computed on upload/edit, anchored to a permissioned blockchain ledger — makes tampering detectable/provable without storing raw documents on-chain.

Audit & Compliance Service

Immutable, append-only audit log of every action: view, download, edit, share, delete-attempt, login.

Reports for compliance (e.g., chain-of-custody reports for evidence).

Search & AI Service

OCR (Tesseract/AWS Textract) for scanned/paper documents.

NLP-based auto-classification and entity extraction (names, dates, sections of law) to auto-tag documents.

Full-text + semantic search via Elasticsearch/OpenSearch.

Case Management Service

Groups documents under a Case ID; tracks case lifecycle (FIR → investigation → chargesheet → trial → judgment).

Workflow/state machine for approvals (e.g., chargesheet review before court filing).

Collaboration Service

Secure sharing across departments (police ↔ prosecution ↔ court) with time-bound, revocable access links.

Comments/annotations scoped to authorized users only.

Notification & Workflow Service

Alerts for pending approvals, court dates, document expiry, SLA breaches.

4. Data Flow (Typical: FIR Upload → Court Filing)

Officer logs in (MFA) → uploads FIR scan via web/mobile.

Document Service stores encrypted file in object storage; metadata written to PostgreSQL.

OCR/NLP pipeline (async via Kafka) extracts text, auto-tags case number, entities.

Integrity Service computes hash, digitally signs, anchors hash to blockchain ledger.

Audit Service logs the "upload" event immutably.

Case Management links the FIR to a new/existing Case ID; workflow state = "Under Investigation."

As investigation proceeds, witness statements/forensic reports are added the same way — each version hashed and logged.

When ready, Investigating Officer submits chargesheet → workflow triggers review/approval by senior officer → routed to prosecution.

Court clerk retrieves case bundle via role-scoped search; every access is audited.

At any point, integrity of any document can be verified by recomputing its hash and comparing to the blockchain-anchored value — proving it hasn't been tampered with (critical for evidentiary admissibility).

5. Recommended Tech Stack

Layer	Technology
Backend framework	Spring Boot (Spring MVC/WebFlux), Spring Security, Spring Cloud Gateway, Spring Data JPA
Auth	OAuth2/OIDC (Keycloak), JWT, MFA (TOTP)
Database (metadata)	PostgreSQL
Document storage	MinIO or AWS S3 (server-side encryption)
Cache/session	Redis
Search	Elasticsearch / OpenSearch
Messaging/async	Apache Kafka
OCR/AI	Tesseract OCR or AWS Textract; spaCy/HuggingFace transformers for NLP tagging
Blockchain (integrity ledger)	Hyperledger Fabric (permissioned) — stores document hashes, not raw content
Digital signatures	PKI via Bouncy Castle / DocuSign-style eSignature integration
Frontend	React or Angular
Infra/deployment	Docker, Kubernetes, CI/CD (Jenkins/GitHub Actions)
Monitoring	Prometheus + Grafana, ELK stack for logs
Cloud	AWS/Azure/GCP (or on-prem/government cloud for data residency compliance)

6. Security & Compliance Measures

Encryption at rest (AES-256) and in transit (TLS 1.3).

RBAC/ABAC with least-privilege defaults.

Blockchain-anchored hashes for tamper-evidence (not full documents on-chain, for cost/privacy).

Digital signatures for legal non-repudiation.

Immutable audit trail (append-only log, ideally write-once storage or log-anchoring to blockchain too).

Data residency/compliance with local laws (e.g., IT Act 2000, Evidence Act provisions on electronic records in India, if applicable).

Regular penetration testing and vulnerability scanning.

7. Feature-to-Requirement Coverage Summary

Problem Statement	Requirement	Feature(s) That Address It	Covered?
Difficulty locating documents quickly		Elasticsearch full-text/semantic search + AI auto-tagging + Case Management grouping	 Yes
Unauthorized access to confidential information		IAM (RBAC/ABAC, MFA, OAuth2/OIDC), encryption at rest/in transit	 Yes
Document tampering risks		Digital signatures + blockchain hash-anchoring + version control	 Yes
Lack of version control		Document Service versioning (immutable version history)	 Yes
Inefficient collaboration between departments		Collaboration Service (secure, time-bound sharing across police/prosecution/court)	 Yes
Delays in legal and investigative processes		Case Management workflow engine + Notification/Workflow Service	 Yes
Poor auditability and compliance tracking		Audit & Compliance Service (immutable logs, chain-of-custody reports)	 Yes
Digitize and centralize document storage		Document Service + centralized object storage + OCR for paper records	 Yes
Ensure secure access and confidentiality		IAM + encryption	 Yes
Prevent unauthorized modifications		Digital signatures + blockchain integrity + RBAC write permissions	 Yes
Maintain complete audit trail		Audit & Compliance Service	 Yes
Enable efficient search and retrieval		Search & AI Service	 Yes
Support collaboration among stakeholders		Collaboration Service	 Yes
Ensure compliance with legal/regulatory requirements		Compliance reporting, digital signature legal validity, data residency controls	 Yes

Problem Statement Requirement	Feature(s) That Address It	Covered?
Preserve evidentiary integrity / legal validity	Blockchain hash-anchoring + digital signatures + chain-of-custody audit reports	☒ Yes

Gaps / things to explicitly decide before build:

Which specific legal framework governs digital signatures/electronic evidence in your jurisdiction (affects signature/cert provider choice).

Data residency requirement — does this need to run entirely on-prem/government cloud, or is public cloud acceptable?

Retention & disposal policy for closed cases (legal hold requirements).

Whether blockchain is a hard requirement or whether a simpler tamper-evident hash-chain/WORM storage log would meet the same goal more cheaply — worth validating against your actual compliance mandate before committing to Hyperledger Fabric's operational overhead.

8. Suggested Build Order (Phases)

Phase 1 — Core: IAM, Document Service (upload/versioning/encryption), PostgreSQL + object storage, basic Case Management.

Phase 2 — Trust layer: Digital signatures, hash computation, audit logging.

Phase 3 — Intelligence: OCR, NLP tagging, Elasticsearch search.

Phase 4 — Integrity ledger: Blockchain hash-anchoring, chain-of-custody reporting.

Phase 5 — Collaboration & workflow: Cross-department sharing, notifications, approval workflows.

Phase 6 — Hardening: Pen-testing, compliance audit, load testing, monitoring/observability.